

Template Engines — Server-Side Rendering with EJS

Section: 9 — Views, Templates, and Static Assets

Subtopic: 01 — Template Engines

Estimated Duration: 2.5–3 hours

Theory

Practical

Topics covered: Server-side rendering vs client-side rendering, what template engines solve, EJS installation and setup, `app.set('view engine')`, `res.render()`, passing data to views, EJS syntax (output tags, control flow, unescaped output), locals and `res.locals`, overview of Pug and Handlebars

Learning Objectives — This Subtopic

- Explain what server-side rendering is and why template engines exist
- Install and configure EJS as the view engine in an Express application
- Render dynamic HTML pages using `res.render()` with data passed from route handlers
- Use EJS syntax for output, control flow (conditionals and loops), and unescaped HTML
- Understand how `res.locals` and `app.locals` make data available to all templates
- Compare EJS, Pug, and Handlebars at a high level to make an informed engine choice

1. The Problem: Serving Dynamic HTML

In Section 6, you built HTTP servers that sent plain text and manually concatenated HTML strings. In Sections 7 and 8, Express routes returned JSON with `res.json()`. Both approaches have a significant limitation when it comes to HTML pages: building HTML by concatenating strings inside JavaScript is error-prone, unreadable, and impossible to maintain at scale.

Consider this route from Section 8:

inline-html-example.js

```
app.get('/users/:id', (req, res) => {
  const user = { name: 'Alice', role: 'Developer', joined: '2024-03-15' };

  res.send(`
    <html>
    <body>
      <h1>${user.name}</h1>
      <p>Role: ${user.role}</p>
      <p>Joined: ${user.joined}</p>
    </body>
    </html>
  `);
});
```

This works for a trivial page, but it falls apart quickly:

- No syntax highlighting for the HTML inside a template literal.
- No way to reuse a common header, footer, or navigation across pages.
- Escaping user input against XSS requires manual effort on every interpolation.
- A full page with dozens of dynamic values becomes an unreadable wall of backticks and `${}`.

Template engines solve all of these problems. They let you write HTML in **separate files** with a small amount of special syntax for injecting dynamic data. The engine compiles the template, merges it with the data you provide, and returns the final HTML string.

Analogy: A template engine works like a mail merge. You have a letter template with placeholders (Dear {{name}}), and a data source with actual values. The engine combines them to produce personalised letters. You write the template once; the data changes per request.

1.1 Server-Side Rendering vs Client-Side Rendering

It is worth distinguishing the two main approaches to rendering HTML:

Aspect	Server-Side Rendering (SSR)	Client-Side Rendering (CSR)
Where HTML is built	On the server, before sending to the browser	In the browser, using JavaScript (React, Vue, etc.)
What the browser receives	Complete HTML document	Minimal HTML shell + JS bundle that builds the DOM
Initial load speed	Faster — browser can render immediately	Slower — must download, parse, and execute JS first
SEO	Good — crawlers see full content	Requires additional work (SSR frameworks, pre-rendering)
Interactivity	Requires full page reloads or added client-side JS	Highly interactive single-page experience

Template engines are a server-side rendering tool. They are the right choice for content-heavy pages, admin panels, multi-page applications, and situations where SEO or fast initial paint matters more than rich client-side interactivity.

2. Setting Up EJS in Express

Express supports multiple template engines. This section focuses on **EJS (Embedded JavaScript)** because its syntax is closest to standard HTML — if you know HTML, EJS templates look immediately familiar. Pug and Handlebars are covered in an overview at the end.

Create a new project:

```
mkdir views-demo && cd views-demo
```

```
npm init -y
```

```
npm install express ejs
```

Stage 1 Minimal EJS setup

```
views-demo/  
├─ node_modules/  
├─ views/  
│   └─ home.ejs  
├─ app.js  
└─ package.json
```

app.js

```
const express = require('express');  
const path = require('path');  
  
const app = express();  
  
// Tell Express which template engine to use  
app.set('view engine', 'ejs');  
  
// Tell Express where to find template files  
app.set('views', path.join(__dirname, 'views'));  
  
app.get('/', (req, res) => {  
  res.render('home');  
});  
  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

views/home.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home</title>
</head>
<body>
  <h1>Welcome to the Views Demo</h1>
  <p>This page is rendered by the EJS template engine.</p>
</body>
</html>
```

```
node app.js
```

Open `http://localhost:3000` in a browser.

Output:

Welcome to the Views Demo
This page is rendered by the EJS template engine.

What is happening?

- `app.set('view engine', 'ejs')` registers EJS as the default template engine. Express will automatically look for files with the `.ejs` extension.
- `app.set('views', path.join(__dirname, 'views'))` tells Express where to find template files. The default is already `./views`, but being explicit avoids surprises if the working directory changes.
- `res.render('home')` looks for `views/home.ejs`, compiles it, and sends the resulting HTML as the response. You do not need to include the `.ejs` extension.
- At this stage, the template contains only static HTML — no dynamic data. The next stage adds that.

Key Insight: You only need to `npm install ejs`. You do *not* need to `require('ejs')` anywhere. Express handles the integration internally — the `app.set('view engine', 'ejs')` call is all that is needed.

3. Passing Data to Templates with `res.render()`

The real power of a template engine is injecting data from the server into the HTML. `res.render()` accepts an optional second argument: an object whose properties become variables available inside the template.

Stage 2 Passing data to a template `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/', (req, res) => {
  res.render('home', {
    title: 'Dashboard',
    username: 'Alice',
    notifications: 5
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

`views/home.ejs`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= title %></title>
</head>
<body>
  <h1>Hello, <%= username %></h1>
  <p>You have <%= notifications %> unread notifications.</p>
</body>
</html>
```

```
node app.js
```

Output:

```
Hello, Alice  
You have 5 unread notifications.
```

What is happening?

- The object `{ title: 'Dashboard', username: 'Alice', notifications: 5 }` is the **data object** (sometimes called the **locals**).
- Inside the `.ejs` file, `<%= title %>` outputs the value of `title`. The `<%= %>` tag is the **output tag** — it evaluates the expression and inserts the result into the HTML.
- Each property of the data object becomes a top-level variable in the template. You access `username` directly, not `data.username`.

4. EJS Syntax Reference

EJS uses a small set of special tags embedded in otherwise standard HTML. Every tag begins with `<%=` and ends with `%>`.

Tag	Name	Purpose
<code><%= expression %></code>	Output (escaped)	Evaluates the expression and inserts the result. HTML characters are escaped (<code><</code> becomes <code>&lt;</code>). Use this for all user-provided data.
<code><%- expression %></code>	Output (unescaped)	Inserts the result without escaping. Use only when you trust the content (e.g., including a partial).

<code><% code %></code>	Scriptlet	Executes JavaScript but produces no output. Used for control flow (if, for, etc.).
<code><## comment %></code>	Comment	Ignored entirely — not included in the rendered HTML.
<code>-%></code>	Trim newline	Trims the trailing newline after the tag, reducing whitespace in output.

4.1 Escaped vs Unescaped Output

The difference between `<%= %>` and `<%- %>` is critical for security.

Stage 3 Escaped vs unescaped output `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/profile', (req, res) => {
  res.render('profile', {
    username: '<script>alert("XSS")</script>',
    bio: '<strong>Full-stack developer</strong> who loves Node.js'
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

`views/profile.ejs`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Profile</title>
</head>
<body>
  <h1>Profile</h1>

  <h2>Escaped (safe):</h2>
  <p><%= username %></p>

  <h2>Unescaped (dangerous with untrusted data):</h2>
  <p><%- username %></p>

  <h2>Unescaped (acceptable with trusted HTML):</h2>
  <p><%- bio %></p>
</body>
</html>
```

```
node app.js
```

Open `http://localhost:3000/profile` in a browser and view the page source:

Output:

```
<!-- Escaped output: -->
<p>&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;</p>

<!-- Unescaped output (the script tag is real HTML – it would execute!): -->
<p><script>alert("XSS")</script></p>

<!-- Unescaped trusted HTML (the <strong> tag renders correctly): -->
<p><strong>Full-stack developer</strong> who loves Node.js</p>
```

What is happening?

- `<%= username %>` escaped the angle brackets, converting `<script>` to `<script>`. The browser renders it as literal text. No script executes.
- `<%- username %>` inserted the raw string. If this were a real page, the `<script>` tag would execute — a textbook XSS vulnerability.

- `<%- bio %>` is acceptable because `bio` is server-controlled content that intentionally contains HTML markup.

Warning: Always use `<%= %>` (escaped output) for any data that originates from user input, query parameters, databases, or external APIs. Only use `<%- %>` when the content is generated and controlled by your own server code (such as including partials — covered in Section 09-03).

5. Control Flow in Templates

The scriptlet tag `<% %>` runs JavaScript without producing output. This is how you add conditionals and loops to templates.

5.1 Conditionals

Stage 4 Conditional rendering `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/dashboard', (req, res) => {
  res.render('dashboard', {
    username: 'Alice',
    isAdmin: true,
    notifications: []
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

views/dashboard.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dashboard</title>
</head>
<body>
  <h1>Dashboard</h1>
  <p>Welcome, <%= username %></p>

  <% if (isAdmin) { %>
    <div style="background: #e8f5e9; padding: 10px; border-radius: 4px;">
      <strong>Admin Panel</strong>
      <p>You have access to admin tools.</p>
    </div>
  <% } %>

  <% if (notifications.length > 0) { %>
    <p>You have <%= notifications.length %> notifications.</p>
  <% } else { %>
    <p>No new notifications.</p>
  <% } %>
</body>
</html>
```

node app.js

Output:

Dashboard
Welcome, Alice

Admin Panel
You have access to admin tools.

No new notifications.

What is happening?

- `<% if (isAdmin) { %>` opens a JavaScript `if` block. The HTML between this tag and `<% } %>` is only included in the output if the condition is true.

- The opening brace `{` and closing brace `}` are placed inside their own `<% %>` tags. The HTML sits between them as regular markup.
- The `else` branch works identically: `<% } else { %>`.
- Since `notifications` is an empty array, `notifications.length > 0` is false, so the "No new notifications" paragraph renders.

5.2 Loops

Stage 5 Looping over arrays

Update the route to pass a list of products:

app.js

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.get('/products', (req, res) => {
  const products = [
    { id: 1, name: 'Mechanical Keyboard', price: 89.99, inStock: true },
    { id: 2, name: 'USB-C Hub', price: 34.50, inStock: true },
    { id: 3, name: 'Monitor Arm', price: 120.00, inStock: false },
    { id: 4, name: 'Desk Lamp', price: 45.00, inStock: true }
  ];

  res.render('products', {
    title: 'Product Catalogue',
    products
  });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

views/products.ejs

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= title %></title>
  <style>
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
    th { background-color: #f6f8fa; }
    .out-of-stock { color: #d73a49; font-weight: bold; }
    .in-stock { color: #28a745; font-weight: bold; }
  </style>
</head>
<body>
  <h1><%= title %></h1>

  <% if (products.length === 0) { %>
    <p>No products available.</p>
  <% } else { %>
    <table>
      <thead>
        <tr>
          <th>#</th>
          <th>Name</th>
          <th>Price</th>
          <th>Availability</th>
        </tr>
      </thead>
      <tbody>
        <% products.forEach((product, index) => { %>
          <tr>
            <td><%= index + 1 %></td>
            <td><%= product.name %></td>
            <td>&pound;<%= product.price.toFixed(2) %></td>
            <td>
              <% if (product.inStock) { %>
                <span class="in-stock">In Stock</span>
              <% } else { %>
                <span class="out-of-stock">Out of Stock</span>
              <% } %>
            </td>
          </tr>
        <% }); %>
      </tbody>
    </table>
    <p><strong>Total products:</strong> <%= products.length %></p>
  </body>
</html>

```

```
<% } %>
</body>
</html>
```

```
node app.js
```

Output:

Product Catalogue

#	Name	Price	Availability
1	Mechanical Keyboard	£89.99	In Stock
2	USB-C Hub	£34.50	In Stock
3	Monitor Arm	£120.00	Out of Stock
4	Desk Lamp	£45.00	In Stock

Total products: 4

What is happening?

- `<% products.forEach((product, index) => { %>` opens a `forEach` loop. Every HTML element between this tag and the closing `<% } %>` is repeated once per product.
- Inside the loop, `<%= product.name %>` outputs each product's name. Standard JavaScript expressions work inside `<%= %>` tags — `product.price.toFixed(2)` formats the price to two decimal places.
- The conditional inside the loop (`if (product.inStock)`) toggles between two `<span>` elements with different CSS classes.
- A `for...of` loop would work equally well: `<% for (const product of products) { %>`.

Key Insight: Any valid JavaScript runs inside `<% %>` tags. However, keep template logic to a minimum — conditionals and loops for display purposes. Business logic (filtering, sorting, computation) belongs in the route handler. The template should receive pre-processed data and focus on presentation.

6. Locals — Sharing Data Across All Templates

Some data is needed on every page: the application name, the current year for a copyright footer, the logged-in user's name, flash messages. Passing these in every `res.render()` call is repetitive. Express provides two mechanisms to avoid this.

6.1 `app.locals`

`app.locals` is an object whose properties are available in **every template** for the lifetime of the application. Set it once, and all templates can access the values.

Stage 6 Using `app.locals` `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Available in every template, for every request
app.locals.appName = 'My Application';
app.locals.currentYear = new Date().getFullYear();

app.get('/', (req, res) => {
  res.render('home', {
    username: 'Alice'
  });
});

app.get('/about', (req, res) => {
  res.render('about');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

`views/home.ejs`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= appName %> - Home</title>
</head>
<body>
  <h1><%= appName %></h1>
  <p>Hello, <%= username %></p>
  <footer>&copy; <%= currentYear %> <%= appName %></footer>
</body>
</html>
```

views/about.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= appName %> - About</title>
</head>
<body>
  <h1>About <%= appName %></h1>
  <p>This is a demo application built with Express and EJS.</p>
  <footer>&copy; <%= currentYear %> <%= appName %></footer>
</body>
</html>
```

node app.js

Visit `http://localhost:3000` :

Output:

```
My Application
Hello, Alice
© 2026 My Application
```

Visit `http://localhost:3000/about` :

Output:

About My Application
This is a demo application built with Express and EJS.
© 2026 My Application

Both pages access `appName` and `currentYear` without either route needing to pass them explicitly.

6.2 `res.locals`

`res.locals` works the same way, but is scoped to a **single request**. It is typically set in middleware and is the standard way to pass request-specific shared data (e.g., the authenticated user).

Stage 7 Using `res.locals` in middleware `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.locals.appName = 'My Application';
app.locals.currentYear = new Date().getFullYear();

// Middleware: simulate authentication
app.use((req, res, next) => {
  // In a real app, this would come from a session or JWT
  res.locals.user = {
    name: 'Alice',
    role: 'admin'
  };
  next();
});

app.get('/', (req, res) => {
  res.render('home-locals');
});

app.get('/settings', (req, res) => {
  res.render('settings');
});
```

```
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');  
});
```

views/home-locals.ejs

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title><%= appName %></title>  
</head>  
<body>  
  <nav>Logged in as: <%= user.name %> (<%= user.role %>)</nav>  
  <h1>Welcome back, <%= user.name %></h1>  
</body>  
</html>
```

views/settings.ejs

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Settings - <%= appName %></title>  
</head>  
<body>  
  <nav>Logged in as: <%= user.name %> (<%= user.role %>)</nav>  
  <h1>Settings</h1>  
  <p>Account: <%= user.name %></p>  
</body>  
</html>
```

node app.js

Output:

```
// http://localhost:3000  
Logged in as: Alice (admin)  
Welcome back, Alice  
  
// http://localhost:3000/settings
```

Logged in as: Alice (admin)
 Settings
 Account: Alice

What is happening?

- The middleware runs on every request and sets `res.locals.user`. This value is automatically available in every template rendered during that request.
- `res.locals` properties and `res.render()` data properties are merged. If both define the same key, the `res.render()` value takes precedence.
- This pattern is how real applications make the authenticated user available to navigation bars, sidebars, and footers without repeating the data in every route.

Mechanism	Scope	Best For
<code>res.render(name, data)</code>	Single template render	Page-specific data (products, posts, form values)
<code>res.locals</code>	Single request	Request-specific shared data (authenticated user, flash messages)
<code>app.locals</code>	Entire application lifetime	Global constants (app name, year, feature flags)

7. Handling Missing Variables

If a template references a variable that was not passed, EJS throws a `ReferenceError` and the page fails to render. This is a common source of errors.

`views/broken.ejs`

```
<h1><%= pageTitle %></h1>
```

app.js (route without passing pageTitle)

```
app.get('/broken', (req, res) => {  
  res.render('broken'); // pageTitle is not passed  
});
```

Output:

```
ReferenceError: /views/broken.ejs:1  
  pageTitle is not defined
```

There are two strategies to handle this:

Strategy 1: Default values in the template

views/safe.ejs

```
<h1><%= typeof pageTitle !== 'undefined' ? pageTitle : 'Default Title' %></h1>
```

Strategy 2: Always pass a complete data object

app.js

```
app.get('/safe', (req, res) => {  
  res.render('safe', {  
    pageTitle: req.query.title || 'Default Title'  
  });  
});
```

```
curl http://localhost:3000/safe
```

Output:

Default Title

```
curl "http://localhost:3000/safe?title=Custom+Page"
```

Output:

Key Insight: Strategy 2 (handling defaults in the route) is preferred. It keeps templates simple and moves logic to where it belongs. The `typeof` check in templates is a useful escape hatch but should not be the default approach.

8. A Realistic Example — User List Page

This example combines everything covered so far into a more complete page: data from the route, loops, conditionals, `app.locals`, and proper escaping.

Stage 8 Full user list page `app.js`

```
const express = require('express');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.locals.appName = 'Admin Portal';
app.locals.currentYear = new Date().getFullYear();

// Simulated user database
const users = [
  { id: 1, name: 'Alice Chen', email: 'alice@example.com', role: 'admin', active: true },
  { id: 2, name: 'Bob Smith', email: 'bob@example.com', role: 'editor', active: true },
  { id: 3, name: 'Charlie Diaz', email: 'charlie@example.com', role: 'viewer', active: false },
  { id: 4, name: 'Diana Patel', email: 'diana@example.com', role: 'editor', active: true }
];

app.get('/users', (req, res) => {
  const roleFilter = req.query.role;

  const filteredUsers = roleFilter
    ? users.filter(u => u.role === roleFilter)
```

```

: users;

res.render('user-list', {
  title: 'User Management',
  users: filteredUsers,
  activeFilter: roleFilter || 'all',
  totalCount: users.length
});
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

views/user-list.ejs

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title><%= title %> - <%= appName %></title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 800px; margin: 40px auto; padding: 0
20px; }
    table { border-collapse: collapse; width: 100%; margin-top: 16px; }
    th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
    th { background-color: #f6f8fa; }
    .badge { display: inline-block; padding: 2px 8px; border-radius: 12px; font-size:
0.8em; font-weight: 600; }
    .badge-admin { background: #e3f2fd; color: #1565c0; }
    .badge-editor { background: #e8f5e9; color: #2e7d32; }
    .badge-viewer { background: #f3e5f5; color: #7b1fa2; }
    .status-active { color: #28a745; }
    .status-inactive { color: #dc3545; }
    .filters a { margin-right: 12px; }
    footer { margin-top: 40px; color: #6c757d; font-size: 0.9em; }
  </style>
</head>
<body>
  <h1><%= title %></h1>

  <div class="filters">
    <strong>Filter by role:</strong>
    <a href="/users">All (<%= totalCount %>)</a>
    <a href="/users?role=admin">Admin</a>
    <a href="/users?role=editor">Editor</a>

```

```

    <a href="/users?role=viewer">Viewer</a>
  </div>

  <% if (users.length === 0) { %>
    <p>No users match the filter "<%= activeFilter %>".</p>
  <% } else { %>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Role</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        <% users.forEach(user => { %>
          <tr>
            <td><%= user.id %></td>
            <td><%= user.name %></td>
            <td><%= user.email %></td>
            <td>
              <span class="badge badge-<%= user.role %>">
                <%= user.role %>
              </span>
            </td>
            <td>
              <% if (user.active) { %>
                <span class="status-active">Active</span>
              <% } else { %>
                <span class="status-inactive">Inactive</span>
              <% } %>
            </td>
          </tr>
        <% }); %>
      </tbody>
    </table>
    <p>Showing <%= users.length %> of <%= totalCount %> users.</p>
  <% } %>

  <footer>&copy; <%= currentYear %> <%= appName %></footer>
</body>
</html>

```

```
node app.js
```

Visit `http://localhost:3000/users` :

Output:

User Management

Filter by role: All (4) Admin Editor Viewer

ID	Name	Email	Role	Status
1	Alice Chen	alice@example.com	admin	Active
2	Bob Smith	bob@example.com	editor	Active
3	Charlie Diaz	charlie@example.com	viewer	Inactive
4	Diana Patel	diana@example.com	editor	Active

Showing 4 of 4 users.

© 2026 Admin Portal

Visit `http://localhost:3000/users?role=editor` :

Output:

User Management

Filter by role: All (4) Admin Editor Viewer

ID	Name	Email	Role	Status
2	Bob Smith	bob@example.com	editor	Active
4	Diana Patel	diana@example.com	editor	Active

Showing 2 of 4 users.

© 2026 Admin Portal

This example demonstrates how the route handler handles business logic (filtering by query parameter), while the template handles presentation (table rendering, conditional styling, role badges).

9. Other Template Engines: Pug and Handlebars

EJS is not the only option. Express supports any engine that conforms to the `(path, options, callback)` signature. Two other popular choices are Pug and Handlebars.

9.1 Pug (formerly Jade)

Pug uses significant whitespace (indentation) instead of closing tags. It produces very concise templates but has a learning curve if you are accustomed to HTML.

```
npm install pug
```

app.js (Pug setup)

```
app.set('view engine', 'pug');
```

views/home.pug

```
doctype html
html(lang="en")
  head
    meta(charset="UTF-8")
    title= title
  body
    h1 Hello, #{username}
    if notifications > 0
      p You have #{notifications} notifications.
    else
      p No new notifications.
```

The equivalent EJS for this template would be roughly twice as many lines due to explicit closing tags.

9.2 Handlebars (via express-handlebars)

Handlebars enforces **logic-less** templates. It deliberately limits what you can do inside a template, pushing all logic into the route or helpers.

```
npm install express-handlebars
```

app.js (Handlebars setup)

```
const { engine } = require('express-handlebars');

app.engine('handlebars', engine());
app.set('view engine', 'handlebars');
```

```
views/home.handlebars
```

```
<h1>Hello, {{username}}</h1>
{{#if hasNotifications}}
  <p>You have {{notifications}} notifications.</p>
{{else}}
  <p>No new notifications.</p>
{{/if}}
```

9.3 Comparison

Aspect	EJS	Pug	Handlebars
Syntax	Standard HTML with <%= %> tags	Indentation-based, no closing tags	HTML with {{ }} expressions
Learning curve	Low — if you know HTML, you know EJS	Medium — new syntax to learn	Low — minimal syntax
Logic in templates	Full JavaScript	Full JavaScript	Minimal — uses helpers for logic
Output escaping	<%= %> escapes, <%- %> does not	#{} escapes, !{} does not	{{ }} escapes, {{{ }}} does not
Verbosity	Medium	Low (most concise)	Medium
Community	Very popular in Express ecosystem	Large, established community	Popular, especially in legacy projects

Key Insight: For this course, all further examples use EJS. It was chosen because the HTML-like syntax means you spend time learning Express concepts, not a new template language. In production, the choice often comes down to team preference. All three engines are well-maintained and performant.

Summary

Concept	Key Point
Template engines	Compile HTML files with embedded dynamic expressions, replacing string concatenation
Server-side rendering	HTML is built on the server and sent complete to the browser; good for SEO and initial load speed
<code>app.set('view engine', 'ejs')</code>	Registers EJS; Express automatically resolves <code>.ejs</code> files in the views directory
<code>res.render(name, data)</code>	Compiles the named template with the given data object and sends the HTML response
<code><%= %></code>	Escaped output — safe for user-provided data (prevents XSS)
<code><%- %></code>	Unescaped output — only for trusted, server-controlled HTML
<code><% %></code>	Scriptlet — runs JavaScript for control flow (if/else, forEach) without outputting
<code>app.locals</code>	Application-wide variables available in all templates (app name, year)
<code>res.locals</code>	Request-scoped variables set in middleware (authenticated user, flash messages)
EJS vs Pug vs Handlebars	EJS: HTML-like; Pug: concise indentation-based; Handlebars: logic-less with helpers

Interview Questions

1. What problem do template engines solve in Express applications?

Expected answer: Template engines allow you to write HTML in separate files with placeholders for dynamic data, replacing fragile string concatenation. They handle output escaping (XSS prevention), support reusable layouts and partials, and separate presentation from business logic.

2. How does Express know which template engine to use?

Expected answer: You call `app.set('view engine', 'ejs')` (or `'pug'`, `'handlebars'`, etc.). Express uses this setting to determine which engine to invoke when `res.render()` is called. The engine must be installed as an npm package.

3. What is the difference between `<%= %>` and `<%- %>` in EJS?

Expected answer: `<%= %>` escapes HTML characters in the output (converting `<` to `<`, etc.), making it safe for user-provided data. `<%- %>` outputs the raw, unescaped string. Using `<%- %>` with untrusted data creates an XSS vulnerability.

4. Explain the difference between `app.locals`, `res.locals`, and the data passed to `res.render()`.

Expected answer: `app.locals` persists for the application's lifetime and is available in all templates (e.g., app name, copyright year). `res.locals` is scoped to a single request and is typically set in middleware (e.g., the authenticated user). The data object passed to `res.render()` is specific to that one render call. All three are merged when the template is compiled, with `res.render()` data taking highest precedence.

5. What happens if a template references a variable that was not passed to `res.render()`?

Expected answer: EJS throws a `ReferenceError` and the page fails to render. To prevent this, either ensure all required variables are passed from the route handler, or use a `typeof` check in the template as a fallback.

6. Why should business logic (filtering, sorting, computation) stay in the route handler rather than in the template?

Expected answer: Separation of concerns. The route handler is responsible for preparing and

processing data; the template is responsible for presentation. Keeping logic out of templates makes them easier to read, test, and maintain. It also prevents performance issues from complex operations running during template compilation.

7. Compare EJS, Pug, and Handlebars. What factors influence the choice between them?

Expected answer: EJS uses standard HTML with embedded JS tags, making it easy for anyone who knows HTML. Pug uses indentation-based syntax (no closing tags) and is more concise but has a steeper learning curve. Handlebars enforces logic-less templates, pushing logic into helpers. The choice typically depends on team familiarity, preference for conciseness vs readability, and whether the team wants to enforce strict separation of logic and templates.

8. In what scenarios is server-side rendering (with template engines) preferable to client-side rendering (with React, Vue, etc.)?

Expected answer: SSR is preferred when SEO is important (search engine crawlers see fully rendered content), when fast initial page load matters, for content-heavy sites with minimal interactivity (blogs, documentation), for admin panels, and when the team does not require a rich single-page application experience. CSR excels at highly interactive, dynamic UIs.

Practical Assignment: Staff Directory with EJS

Objective: Build an Express application that renders a staff directory page with EJS, using `app.locals`, `res.locals`, loops, conditionals, and query-based filtering.

Setup

1. Create a new directory called `staff-directory` and initialise:

```
mkdir staff-directory && cd staff-directory && npm init -y
```

2. Install dependencies:

```
npm install express ejs
```

Requirements

1. Create the following folder structure:

```
staff-directory/  
├─ views/  
│   ├─ staff-list.ejs  
│   └─ staff-detail.ejs  
├─ app.js  
└─ package.json
```

2. Set `app.locals` for `companyName` ('Acme Corp') and `currentYear` .
3. Create a middleware that sets `res.locals.navLinks` to an array of navigation objects:
`[{ label: 'Staff', url: '/staff' }, { label: 'About', url: '/about' }]` .
4. Create an in-memory array of at least 6 staff members, each with: `id` , `name` , `department` ('Engineering' , 'Design' , 'Marketing'), `role` , `email` , `active` (boolean).
5. **GET /staff** — Renders `staff-list.ejs` :
 - Accepts optional query parameter `?department=Engineering` to filter.
 - Displays a table of staff members with name, department, role, and status.
 - Each name links to `/staff/:id` .
 - Shows filter links for each department and "All".
 - Displays a count: "Showing X of Y staff members."
 - Active/inactive status is colour-coded (green/red).
6. **GET /staff/:id** — Renders `staff-detail.ejs` :
 - Displays full details for a single staff member.
 - If the ID does not exist, renders the template with an error message (not a JSON response).
 - Includes a "Back to Staff List" link.
7. Both pages must display a navigation bar built from `res.locals.navLinks` and a footer using `app.locals.companyName` and `currentYear` .

Expected Output — Staff List

Visit <http://localhost:3000/staff> :

Output:

Acme Corp

[Staff](#) | [About](#)

Staff Directory

Filter: All (6) Engineering Design Marketing

Name	Department	Role	Status
Alice Chen	Engineering	Senior Developer	Active
Bob Smith	Design	UI Designer	Active
Charlie Diaz	Engineering	Junior Developer	Inactive
Diana Patel	Marketing	Content Lead	Active
Eve Johnson	Design	UX Researcher	Active
Frank Lee	Engineering	DevOps Engineer	Active

Showing 6 of 6 staff members.

© 2026 Acme Corp

Expected Output — Staff Detail

Visit <http://localhost:3000/staff/1> :

Output:

Acme Corp

[Staff](#) | [About](#)

Alice Chen

Department: Engineering
Role: Senior Developer
Email: alice@acmecorp.com
Status: Active

[← Back to Staff List](#)

© 2026 Acme Corp

Bonus Challenge

Add a `GET /staff/:id/edit` route that renders an EJS form pre-populated with the staff member's current data (name, department, role, email, active). The form should POST to `/staff/:id`. On the POST route, update the in-memory data and redirect back to `/staff/:id`. Use `express.urlencoded()` to parse the form body. This is a preview of how forms, templates, and request handling work together in a server-rendered application.

